
MILESTONE 1 — DATA MODEL SPECIFICATION

FreightQuote AI

MongoDB Collections, Money Handling, Geospatial & Quote Versioning for Weeks 1-2

DOCUMENT TYPE	Data model specification — collections, document shapes, indexes, decisions
PROJECT CODE	FQ-AMB-001 / M1-DATA
DATABASE	MongoDB 7.x (Atlas or self-hosted replica set)
SCOPE	Milestone 1 only — enquiry, gateways, routes, transit, indicative quote
CRITICAL	Section 1 (money) and section 2 (quote immutability) are non-negotiable. Get them wrong and Milestone 2 requires a data migration.

1. Money: Decimal128, Never Double

MongoDB's default number type is an IEEE 754 double. Storing money in it is a defect, not a style choice.

0.1 + 0.2 stored as doubles is 0.30000000000000004. Across a quote with eight cost components, three currencies and an FX conversion, the error compounds into visible discrepancies. A broker who spots a one-rupee mismatch between the line items and the total stops trusting the whole system — and they will spot it.

Layer	Rule
MongoDB	Decimal128 BSON type for every monetary field. No exceptions, including zero values and placeholders.
Python	decimal.Decimal throughout. PyMongo converts Decimal128 ↔ Decimal via bson.decimal128.Decimal128. Never let a monetary value touch float().
API transport	Serialise as a string , not a JSON number: {"amount": "384500.0000", "currency": "INR"}. JSON numbers are parsed as doubles by JavaScript, which undoes everything you just did.
Frontend	Keep as string; format for display only. Never arithmetic in JavaScript on a monetary value.
Precision	4 decimal places stored. Round to 2 only at final display, using one documented policy in one module.

1.1 Every monetary field is a pair

An amount without a currency is meaningless in a multi-currency brokerage. Store them together as an embedded object so they cannot be separated by a careless projection:

```
// correct
freight_cost: { amount: Decimal128("384500.0000"), currency: "INR" }

// wrong – currency lives elsewhere and gets lost
freight_cost: Decimal128("384500.0000")
```

On the Django question

The same constraint from the SupportPilot data model applies here: Django's ORM, migrations, admin and auth are SQL-bound and do not work against MongoDB. FastAPI + Beanie remains the recommendation, and it matters more on this project — Beanie is Pydantic-based, and Pydantic gives you Decimal validation on every monetary field for free.

2. Quote Versions Are Immutable

This is the second decision that cannot be retrofitted. A freight quotation is a commercial document. If a customer disputes a price three months later, you must be able to produce exactly what was sent, computed from exactly the inputs that were live at the time.

Rule	Implementation
Never update a quote version	A price change inserts a new document in <code>quote_versions</code> and marks the previous one <code>SUPERSEDED</code> . The only field ever updated on an existing version is its status.
Freeze every input	FX rate, lane rate reference, model versions, data-source timestamps and a full route snapshot are copied onto the version at creation. Never referenced by ID alone.
Snapshot the route, do not reference it	Routes get regenerated when an enquiry is re-run. If the quote only holds <code>route_id</code> , regenerating routes silently rewrites history behind an issued quote.
Two collections, not one	<code>freight_quotes</code> holds the stable identity (quote number, customer, current version pointer). <code>quote_versions</code> holds the immutable priced documents.

3. Milestone 1 Collections

users

Internal staff and external customer-portal users in one collection, separated by role and customer_id.

```
{
  _id:      ObjectId,
  email:    "ravi.k@portline.com",           // unique
  full_name: "Ravi Kumar",
  password_hash: "$argon2id$...",
  role:     "BROKER",                        // CUSTOMER|BROKER|SENIOR_BROKER|PRICING_MANAGER|
                                              // OPERATIONS|COMPLIANCE_OFFICER|EXECUTIVE|ADMIN|SERVICE
  customer_id: null,                        // SET for CUSTOMER role – scopes every portal query
  team:     "Trade Lane – Middle East",
  is_active: true,
  mfa_enabled: false,
  last_login_at: ISODate,
  created_at: ISODate
}
```

Indexes: {email: 1} unique · {role: 1, is_active: 1} · {customer_id: 1}

The customer_id claim is your tenant boundary

Every portal query must filter on `customer_id` taken from the JWT, never from a request parameter. This is the most likely serious bug in this system: one customer seeing another's quote. Write the IDOR test before the feature.

customers

```
{
  _id:      ObjectId,
  code:     "CUST-00184",                    // unique
  company_name: "Sharma Textiles Pvt Ltd",
  country:   "IN",
  tier:      "KEY",                          // STRATEGIC|KEY|STANDARD|NEW – feeds margin logic (M2)
  default_currency: "INR",
  payment_terms: "NET30",
  account_manager_id: ObjectId,
  contacts: [                               // EMBEDDED – bounded
    { name: "Priya Sharma", email: "priya@sharmatextiles.in",
      phone: "+919840012345", is_primary: true }
  ],
  is_active: true,
  created_at: ISODate
}
```

Indexes: {code: 1} unique · {company_name: "text"} · {tier: 1, is_active: 1}

ports

Ports, airports and rail hubs in one collection. GeoJSON coordinates with a 2dsphere index.

```
{
  _id:      ObjectId,
  code:     "INNSA",                        // UN/LOCODE or IATA – unique
  code_type: "UNLOCODE",                    // UNLOCODE|IATA
  name:     "Nhava Sheva (JNPT)",
  city:     "Mumbai",
  country:  "IN",
  region:   "SOUTH_ASIA",
  type:     "PORT",                         // PORT|AIRPORT|RAIL_HUB
  modes:    ["OCEAN"],                     // which modes this gateway serves

  location: {                               // GeoJSON – required shape for 2dsphere
    type: "Point",
    coordinates: [72.9520, 18.9490]        // [longitude, latitude] – LON FIRST
  },

  timezone: "Asia/Kolkata",
  calendar_code: "IN_MUMBAI",
  is_transshipment_hub: false,
  customs_clearance_hrs: 24,
  cutoff_hrs: 6,
  is_active: true
}
```

GeoJSON coordinate order is [longitude, latitude]

The opposite of how everyone says it and writes it. Reversing them puts Mumbai in the Indian Ocean off Somalia, and the failure is silent — queries return results, just wrong ones. Add an assertion in the port import script that latitude is within ± 90 and longitude within ± 180 , and spot-check three known ports on a map after loading.

Indexes: {code: 1} unique · {location: "2dsphere"} · {type: 1, modes: 1, is_active: 1} · {name: "text", city: "text", code: "text"} for the gateway autocomplete

Nearest-gateway query this enables

```
db.ports.find({
  location: { $near: {
    $geometry: { type: "Point", coordinates: [lon, lat] },
    $maxDistance: 500000 // metres
  }},
  type: "PORT", modes: "OCEAN", is_active: true
}).limit(1)
```

port_distances

Seeded sea distances. The single most important reference table in the system.

```
{
  _id: ObjectId,
  from_code: "INNSA",
  to_code: "AEJEA",
  lane_key: "INNSA-AEJEA", // normalised, unique
  nautical_miles: 1205,
  via: "DIRECT", // DIRECT|SUEZ|PANAMA|CAPE|MALACCA
  chokepoints: [], // ["SUEZ"] etc – feeds M3 risk scoring
  source: "published_table",
  verified_at: ISODate
}
```

Indexes: {lane_key: 1} unique · {from_code: 1, to_code: 1}

Store both directions, or normalise the key

Sea distance is symmetric but your lookups are not — a query for AEJEA→INNSA must find the row. Either insert both directions at import time (simpler, doubles the rows, worth it), or sort the two codes alphabetically to build `lane_key` and sort them the same way at query time. Pick one and enforce it in the import script; a mix of both is how you get intermittent "lane not serviced" errors.

carriers · carrier_services

Services carry the port rotation the route graph is built from.

```
// carriers
{ _id, code: "MAEU", scac: "MAEU", name: "Maersk Line",
  service_type: "OCEAN", reliability_score: Decimal128("0.94"),
  on_time_pct: Decimal128("94.20"), accepts_hazmat: true,
  accepts_reefer: true, is_active: true }

// carrier_services
{
  _id: ObjectId,
  carrier_id: ObjectId,
  carrier_name: "Maersk Line", // denormalised
  service_name: "MECL – Middle East Container Line",
  mode: "OCEAN",
  port_rotation: [ // EMBEDDED, order matters
    { seq: 1, port_code: "INNSA", transit_days_from_prev: 0 },
    { seq: 2, port_code: "AEJEA", transit_days_from_prev: 3 },
    { seq: 3, port_code: "AEAUH", transit_days_from_prev: 1 }
  ],
  sailings_per_week: 1,
  supported_containers: ["20GP", "40GP", "40HC", "40RF"],
  is_active: true
}
```

Indexes: {carrier_id: 1, is_active: 1} · {"port_rotation.port_code": 1, mode: 1, is_active: 1} — a multikey index on the embedded array, which is what lets the Route Agent find every service touching a given port in one query.

container_types

```
{ _id, code: "40HC", name: "40ft High Cube",
  internal_cbm: Decimal128("76.30"), max_payload_kg: 28800,
  is_reefer: false, is_active: true, sort_order: 3 }
```

shipments

The enquiry. Matches the form field spec exactly, including the fields added in M1.

```
{
  _id: ObjectId,
  reference: "SHP-2026-01842", // unique, from counters
  customer_id: ObjectId,
  customer_name: "Sharma Textiles Pvt Ltd", // denormalised for list views
  created_by_id: ObjectId,
```

```

source:      "BROKER",                // BROKER|PORTAL|API|EDI

route: {
  origin_port_code:      "INNSA",
  origin_port_name:      "Nhava Sheva, Mumbai",
  origin_country:        "IN",
  destination_port_code: "AEJEA",
  destination_port_name: "Jebel Ali, Dubai",
  destination_country:   "AE",
  lane_key:              "INNSA-AEJEA",      // indexed – powers lane analytics
  pickup_address: { line1: "...", city: "Tirupur", postal_code: "641604",
                    country: "IN", location: { type: "Point",
                    coordinates: [77.3411, 11.1085] } },
  delivery_address: null
},

service: {
  mode:      "OCEAN",                // OCEAN|AIR|GROUND_RAIL|EXPRESS_AIR
  load_type: "FCL",                  // FCL|LCL – required when OCEAN
  incoterm:  "FOB"                    // EXW|FCA|FOB|CIF|CFR|DAP|DDP
},

items: [                                // EMBEDDED – bounded in practice
  {
    line_no: 1,
    package_type: "CONTAINER",        // drives which fields are present
    commodity_description: "Cotton textile rolls, unbleached",
    hs_code: "5208.11",

    // FCL branch
    container_type: "40HC",
    container_count: 2,
    gross_weight_kg: Decimal128("18400.000"),

    // LCL / air branch – null on the FCL path
    quantity: null, weight_per_unit_kg: null,
    length_cm: null, width_cm: null, height_cm: null,
    is_stackable: null
  }
],

additional: {
  declared_value: { amount: Decimal128("0.0000"), currency: "INR" },
  currency: "INR",
  is_fragile: false,
  is_hazardous: true,
  un_number: "UN1263",
  imo_class: "3",
  msds_file_key: "shipments/2026/08/msds-abc.pdf",
  is_temperature_controlled: false,
  temp_min_c: null, temp_max_c: null,
  needs_insurance: false,
  special_instructions: null
},

ready_date:      ISODate("2026-08-12"),
required_delivery_date: null,

status:      "QUOTED",    // ENQUIRY|QUOTING|QUOTED|QUOTE_FAILED|WON|LOST|
                // EXPIRED|CANCELLED

status_history: [          // EMBEDDED – bounded
  { from: null, to: "ENQUIRY", actor_id: ObjectId, at: ISODate },
  { from: "ENQUIRY", to: "QUOTING", actor_id: null, at: ISODate }
],

created_at: ISODate,
updated_at: ISODate
}

```

Indexes: {reference: 1} unique · {customer_id: 1, created_at: -1} · {"route.lane_key": 1, created_at: -1} · {status: 1, created_at: -1} · {created_by_id: 1, status: 1}

routes

Generated per shipment by the Route Agent. Legs embedded; geometry kept here, not on the quote.

```

{
  _id:      ObjectId,
  shipment_id: ObjectId,
  rank:      1,
  is_recommended: true,

  carrier_id:      ObjectId,
  carrier_name: "Maersk Line",
  service_name: "MECL",

  legs: [          // EMBEDDED – bounded, ordered
    { seq: 1, from_port_code: "INNSA", to_port_code: "AEJEA",
      carrier_name: "Maersk Line", distance_nm: 1205,
      transit_days: Decimal128("3.0"), is_transshipment: false }
  ],

```

```

total_distance_nm: 1205,
transhipment_count: 0,

transit: {                                     // EMBEDDED breakdown – shown in UI
  pickup_days:    Decimal128("1.0"),
  origin_dwell:   Decimal128("3.0"),
  linehaul_days:  Decimal128("3.0"),
  schedule_wait:  Decimal128("3.5"),
  dest_dwell:     Decimal128("3.0"),
  delivery_days:  Decimal128("0.0"),
  total_days:     Decimal128("13.5"),
  range_min_days: 6,
  range_max_days: 10,
  estimated_arrival: ISODate("2026-08-22"),
  model_version:  "transit-v1.2-lgbm"
},

scores: {                                     // shown as the ranking explanation
  transit:    Decimal128("0.92"),
  cost:       Decimal128("0.78"),
  reliability: Decimal128("0.94"),
  congestion: Decimal128("0.71"),
  composite:  Decimal128("0.8595")
},

geometry: {                                   // GeoJSON – for the map
  type: "LineString",
  coordinates: [[72.952,18.949],[70.1,20.4],[58.5,24.2],[55.027,24.985]]
},

data_as_of: { carrier_schedules: ISODate, port_congestion: ISODate },
created_at: ISODate
}

```

Indexes: {shipment_id: 1, rank: 1} · {geometry: "2dsphere"} · TTL on created_at for unselected routes if volume grows

freight_quotes

The stable identity. Small, rarely changes, points at the current version.

```

{
  _id:           ObjectId,
  quote_number:  "QT-2026-00934",           // unique, from counters
  shipment_id:   ObjectId,
  shipment_reference: "SHP-2026-01842",
  customer_id:   ObjectId,
  customer_name:  "Sharma Textiles Pvt Ltd",
  lane_key:      "INNSA-AEJEA",
  current_version: 1,
  current_version_id: ObjectId,
  status:         "DRAFT", // DRAFT|PENDING_APPROVAL|APPROVED|ISSUED|
                        // ACCEPTED|DECLINED|EXPIRED
  owner_id:       ObjectId, // broker
  created_at:     ISODate,
  updated_at:     ISODate
}

```

Indexes: {quote_number: 1} unique · {shipment_id: 1} · {customer_id: 1, created_at: -1} · {status: 1, created_at: -1} · {lane_key: 1, created_at: -1} · {owner_id: 1, status: 1}

quote_versions

IMMUTABLE. Insert only. The route is snapshotted, not referenced.

```

{
  _id:           ObjectId,
  quote_id:       ObjectId,
  quote_number:  "QT-2026-00934",
  version:       1, // unique with quote_id

  // ---- ROUTE SNAPSHOT – copied, not referenced ----
  route_snapshot: {
    route_id:      ObjectId, // provenance only
    carrier_name:  "Maersk Line",
    service_name:  "MECL",
    legs: [ { seq:1, from:"INNSA", to:"AEJEA", transit_days: Decimal128("3.0"),
              is_transhipment:false } ],
    total_distance_nm: 1205,
    transit_days_min: 6,
    transit_days_max: 10,
    estimated_arrival: ISODate("2026-08-22"),
    transit_breakdown: { pickup: Decimal128("1.0"), origin_dwell: Decimal128("3.0"),
                        linehaul: Decimal128("3.0"), schedule_wait: Decimal128("3.5"),
                        dest_dwell: Decimal128("3.0"), delivery: Decimal128("0.0") }
  },

  // ---- CHARGE BASIS – three mutually exclusive shapes ----
  charge_basis: {

```

```

    type: "PER_CONTAINER", // PER_CONTAINER | REVENUE_TON | CHARGEABLE_KG
    units: Decimal128("2"),
    unit_label: "2 × 40HC",
    // populated only on the CHARGEABLE_KG path:
    actual_weight_kg: null,
    volumetric_weight_kg: null,
    chargeable_weight_kg: null,
    // populated only on the REVENUE_TON path:
    volume_cbm: null, weight_tonnes: null
  },

  currency: "INR",
  fx_rate: Decimal128("1.000000"), // FROZEN
  fx_rate_date: ISODate("2026-08-05"), // FROZEN

  // ---- M1: indicative only. Real build-up arrives in M2 ----
  pricing: {
    is_indicative: true,
    lane_rate_ref: ObjectId, // FROZEN reference
    base: { amount: Decimal128("284814.8148"), currency: "INR" },
    flat_uplift_pct: Decimal128("35.000"),
    final_quote: { amount: Decimal128("384500.0000"), currency: "INR" },
    // M2 adds: cost_components[], surcharges[], margin, total_cost
    cost_components: [],
    total_cost: null,
    margin_pct: null
  },

  valid_until: ISODate("2026-08-19T23:59:59Z"),

  // ---- FROZEN PROVENANCE – makes the quote reproducible ----
  model_versions: { transit: "transit-v1.2-lgbm", route_scorer: "rules-v1.0" },
  data_as_of: { carrier_schedules: ISODate, port_congestion: ISODate,
    port_distances: ISODate, fx: ISODate },
  assumptions: [
    "Rate valid for the stated container type and count only",
    "Excludes duties, taxes and customs clearance",
    "Subject to space and equipment availability",
    "Indicative pricing – final rate confirmed by your account manager"
  ],
  rationale: {
    route: "Direct weekly service selected: highest composite score (0.86), " +
      "3 days faster than the Salalah transshipment alternative.",
    transit: "13.5 days computed; presented as 6–10 day range after buffer.",
    pricing: "Indicative – Milestone 1 flat lane table, no contract rate applied."
  },

  status: "DRAFT", // DRAFT|SUPERSEDED – set once, then frozen
  created_by_id: ObjectId,
  created_at: ISODate
}

```

Indexes: {quote_id: 1, version: -1} · {quote_id: 1, version: 1} unique · {status: 1, valid_until: 1} for the expiry sweep

lane_rates

Milestone 1 stub only. Replaced by rate_cards + rate_card_lines + surcharges in M2.

```

{
  _id: ObjectId,
  origin_region: "SOUTH_ASIA",
  dest_region: "MIDDLE_EAST",
  mode: "OCEAN",
  load_type: "FCL",
  container_type: "40HC",
  per_container: { amount: Decimal128("142407.4074"), currency: "INR" },
  per_kg: null,
  per_rt: null,
  minimum: { amount: Decimal128("85000.0000"), currency: "INR" },
  valid_from: ISODate, valid_to: ISODate,
  source: "brokerage_historical_average",
  is_active: true
}

```

Indexes: {origin_region: 1, dest_region: 1, mode: 1, container_type: 1, is_active: 1}

fx_rates · business_calendars · counters · activity_logs

```

// fx_rates – one document per day per pair
{ _id, base: "INR", quote: "USD", rate: Decimal128("0.011950"),
  rate_date: ISODate("2026-08-05"), source: "ecb" }
// index: {base:1, quote:1, rate_date:-1}

// business_calendars – per country/port
{ _id, code: "IN_MUMBAI", timezone: "Asia/Kolkata",
  working_days: [1,2,3,4,5], work_start: "09:00", work_end: "18:00",
  holidays: [ { date: ISODate("2026-08-15"), name: "Independence Day" } ] }

// counters – atomic sequence generation

```

```
{ _id: "shipment_2026", seq: 1842 }
{ _id: "quote_2026",    seq: 934  }

// activity_logs - append-only
{ _id, actor_id, actor_name, action: "QUOTE_VERSION_CREATED",
  entity_type: "QUOTE", entity_id: ObjectId,
  before: null, after: { version: 1, final_quote: "384500.0000" },
  ip, request_id, created_at: ISODate }
```

Reference numbers must use atomic \$inc

`findOneAndUpdate({_id: "quote_2026"}, {$inc: {seq: 1}}, {returnDocument: "after"})`. Counting existing documents and adding one produces duplicates under concurrency. Back it with a unique index on `quote_number` as a second line of defence.

4. Embedding Decisions Summary

Data	Decision	Reasoning
Shipment line items	Embed	Bounded (rarely more than 10), always read with the shipment, never queried independently.
Route legs	Embed	Bounded at 3-4, meaningless without the route.
Carrier port rotation	Embed	Bounded; the multikey index on it is what makes route graph queries fast.
Transit breakdown & scores	Embed	One per route, always displayed together.
Route snapshot in a quote version	Embed	Immutability requires it. A reference would let regenerated routes rewrite an issued quote.
Customer contacts	Embed	Handful per customer, always read with them.
Status history	Embed	Bounded state machine, shown on the detail timeline.
Route options	Reference	Geometry is bulky and 3 per enquiry accumulates. Also queried independently on the route detail page.
Quote versions	Reference	Unbounded growth and each must be independently addressable for audit.
Ports, carriers, rates, calendars	Reference	Shared master data, edited independently.

5. Transactions & Consistency

Operation	Approach
Create shipment	Increment counter, insert shipment. Transaction — requires a replica set, so configure one even locally.
Persist route options	insertMany on routes, then one updateOne on the shipment status. Transaction, so a half-generated set never appears.
Create quote version	Insert into quote_versions, update freight_quotes.current_version and current_version_id. Transaction — a pointer to a nonexistent version breaks the detail page.
Supersede a version	Set previous version's status to SUPERSEDED, insert the new one, repoint the parent. One transaction, three writes.
Status transition	Conditional update: updateOne({_id, status: expectedFrom}, {...}). If modifiedCount is 0 the transition raced — return 409. Optimistic concurrency without a transaction.
Expiry sweep	updateMany({status:"ISSUED", valid_until:{<: now}}, {\$set:{status:"EXPIRED"}}). Idempotent, safe to re-run.

6. Schema Validation

Apply `$jsonSchema` validators on creation. The monetary type check is the one that earns its keep — it catches a `double` slipping in before it reaches a customer-facing number.

```
db.createCollection("quote_versions", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["quote_id", "quote_number", "version", "currency", "fx_rate",
              "pricing", "charge_basis", "created_at"],
    properties: {
      version: { bsonType: "int", minimum: 1 },
      currency: { enum: ["INR", "USD", "EUR", "AED"] },
      fx_rate: { bsonType: "decimal" },           // rejects double
      charge_basis: { bsonType: "object", properties: {
        type: { enum: ["PER_CONTAINER", "REVENUE_TON", "CHARGEABLE_KG"] }
      } },
      pricing: { bsonType: "object", properties: {
        is_indicative: { bsonType: "bool" },
        final_quote: { bsonType: "object", required: ["amount", "currency"],
          properties: { amount: { bsonType: "decimal" } } }
      } }
    }
  } },
  validationLevel: "strict", validationAction: "error"
})
```

7. Query Patterns to Build Against

Screen	Query shape
Gateway autocomplete	Text search on ports with type and modes filters, limit 10. Sub-50 ms with the text index.
Route Agent — find services	find({"port_rotation.port_code": {\$all: [origin, dest]}}, mode, is_active: true)) using the multikey index, then verify rotation order in application code.
Quotation dashboard	find({...filters}).sort({created_at: -1}).limit(25) with cursor pagination on created_at plus _id. Never skip().
Quote detail	One findOne on freight_quotes, one on quote_versions by current_version_id, one find on routes. Three queries; the route snapshot means the version alone renders the whole commercial view.
Customer portal list	find({customer_id: fromToken, status: {\$in: ["ISSUED", "ACCEPTED", "EXPIRED"]}}) through a customer-only projection that excludes pricing.total_cost, pricing.margin_pct and pricing.cost_components.
Lane performance	aggregate grouping quote_versions by lane_key. Fine at M1 volume; pre-computed rollups in M4.
Expiring soon	find({status: "ISSUED", valid_until: {\$lt: in48h}}) on the compound index.

The customer projection must live in the backend

Define one `CustomerQuoteProjection` and make it the only projection a portal endpoint may use. Write an automated test asserting that no portal response payload ever contains `total_cost`, `margin_pct`, `cost_components` or carrier buy-rate keys. Hiding a field in React while the API still returns it is a commercial data leak, and the network tab is one click away.

8. What Milestone 1 Does NOT Need

Collection	Milestone
rate_cards, rate_card_lines, surcharges, cost_components	M2 — real pricing engine
margin_policies, approval_rules, quote_approvals	M2 — margin and approval
weather_risks, shipment_risk_scores, alerts	M3 — weather and risk
customs_requirements, regulations, regulation_chunks, hs_codes	M3 — customs (regulation_chunks gets the Atlas Vector Search index)
agent_runs, agent_steps	M3-M4 — orchestration traces
port_congestion_snapshots, rate_history	M3-M4 — time series, use time-series collections
daily_quote_rollups, lane_performance_rollups	M4 — analytics

9. Setup Checklist

☐	Item
☐	Framework decision recorded — Django ORM does not work against MongoDB (see the SupportPilot data model, section 1)
☐	MongoDB running as a replica set, including locally — transactions require it
☐	Every monetary field declared as <code>Decimal128</code> ; a CI test asserts no <code>double</code> exists in any amount field
☐	API serialises monetary values as strings, verified by a contract test
☐	Port import asserts coordinate order and range; three known ports spot-checked on a map
☐	2dsphere indexes created on <code>ports.location</code> and <code>routes.geometry</code>
☐	Port distances loaded for the top 50 lanes, both directions (or normalised keys, enforced in import)
☐	Quote versions are insert-only; a test asserts an existing version cannot be updated except for status
☐	Reference numbers use atomic <code>\$inc</code> ; concurrency-tested with 100 simultaneous creates
☐	Customer projection test asserts no cost or margin key appears in any portal response
☐	All indexes verified with <code>explain()</code> showing <code>IXSCAN</code> , not <code>COLLSCAN</code>
☐	Backup configured and one restore rehearsed before the milestone review